DevOps CT2

Scenario Questions with Model Answers

8 fully written 15-mark answers | Unit 3 (Q1-Q4) | Unit 4 (Q5-Q8)

How to use these answers

- Each answer follows the same shape: **introduction, diagram, explanation/steps, code or commands, benefits, conclusion**. Marks are shown in orange brackets - a suggested split, not an official scheme.
- In the exam you get Either/Or: pair them as Q1/Q2 (IaC + Terraform), Q3/Q4 (Ansible), Q5/Q6 (Docker + Swarm), Q7/Q8 (Kubernetes) and attempt the one whose diagram you can draw from memory.
- Write **the scenario words** into your answer (e.g. "for the 50 Ubuntu servers...") - scenario questions reward application.
- Change names and numbers if the exam scenario differs; the structure stays the same.

QUESTION 1 | 15 MARKS | UNIT 3 - IaC

A start-up creates servers manually. Development, testing and production behave differently and releases often fail. Propose an Infrastructure as Code solution using Terraform and Ansible, with a neat diagram.

1. Introduction and problem analysis [2 M]

Infrastructure as Code (IaC) is managing and provisioning infrastructure (servers, networks, load balancers) through machine-readable code that is version-controlled and run by automation tools, instead of manual configuration. In the given scenario, manual setup causes human error, **configuration drift** between environments, slow scaling and no audit trail. IaC removes all four.

2. Diagram [4 M]

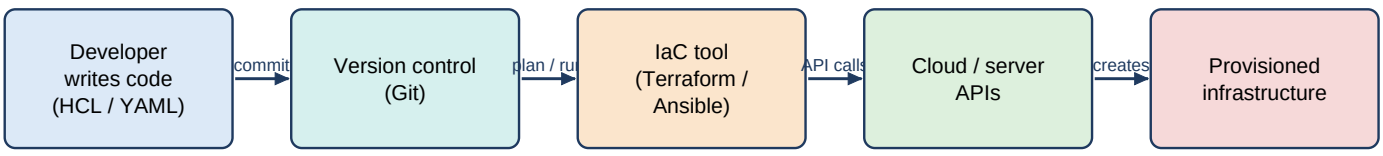


Fig 3.1 Generic Infrastructure-as-Code workflow

3. Proposed solution - step by step [5 M]

Step	Action	Tool
1	Write infrastructure (VMs, network, load balancer) as declarative code; store in Git; review with pull requests.	Terraform + Git
2	Run `terraform plan` to preview additions/modifications/deletions, then `terraform apply`. State file records what exists.	Terraform
3	Terraform outputs server IPs; they are added to the Ansible inventory (static or dynamic).	Terraform output / inventory
4	Ansible playbooks (agentless, over SSH) install packages, copy configuration and start services - idempotent, so safe to re-run.	Ansible
5	Same code is reused for dev, test and prod using different variable files, so all environments are identical.	tfvars / group_vars

Why both tools? Terraform provisions and manages infrastructure (declarative, multi-cloud, state file); Ansible configures servers and deploys applications (agentless, YAML). Together they cover provisioning, configuration and deployment. Model: **push** - a central controller pushes changes.

4. Benefits [3 M]

- **Consistency** - same code gives same environment; drift disappears.
- **Version control** - every change tracked, reviewed, reversible.
- **Scalability and speed** - change a number, apply, get more servers.
- **Automation** - fewer human errors, faster releases.
- **Collaboration and compliance** - shared, auditable standards.

5. Conclusion [1 M]

Adopting IaC with Terraform (provisioning) and Ansible (configuration/deployment) makes the start-up's environments reproducible and its releases predictable.

QUESTION 2 | 15 MARKS | UNIT 3 - Terraform

A team wants to launch an AWS EC2 web server and display its public IP. Explain how Terraform achieves this using input variables and output values. Write the configuration and explain each command.

1. Introduction [2 M]

Terraform (HashiCorp) is an open-source declarative IaC tool. You describe the desired infrastructure in **HCL**; Terraform compares it with the **state file** and calls the cloud **API** through a **provider** plugin to make reality match. It is cloud-agnostic and modular.

2. Diagrams [3 M]

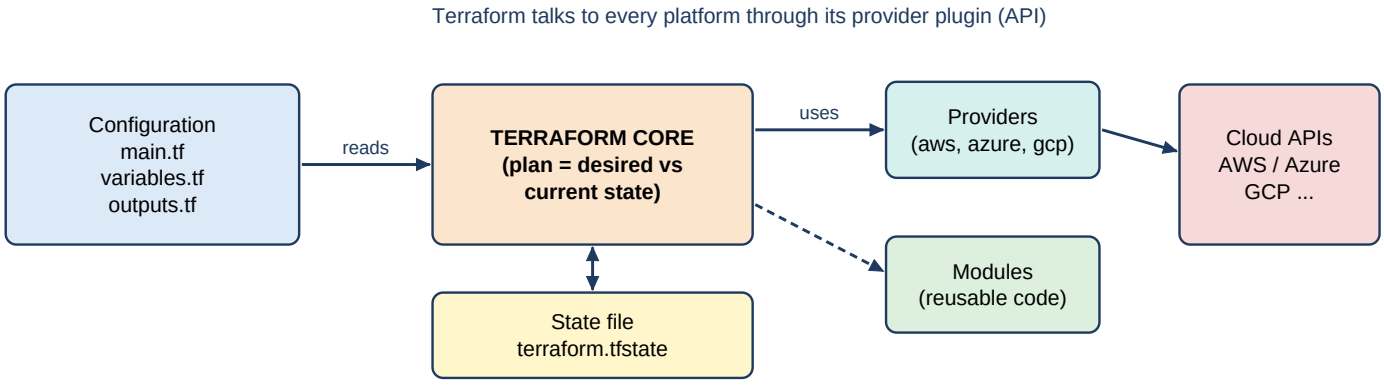


Fig 3.3 Terraform architecture

Fig 3.4 Terraform core workflow (write -> init -> validate -> plan -> apply -> destroy)

3. Configuration files [4 M]

```
# variables.tf  - input variables
variable "region" {
  description = "AWS region"
  type        = string
  default     = "us-west-2"
}
variable "instance_type" {
  type    = string
  default = "t2.micro"
}

# main.tf  - provider and resource
provider "aws" {
  region = var.region
}
resource "aws_instance" "web" {
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = var.instance_type
  tags = {
    Name = "web-server"
  }
}

# outputs.tf  - output variable
output "web_public_ip" {
  value = aws_instance.web.public_ip
}
```

- **provider** selects the cloud and region; **resource "aws_instance" "web"** declares one VM (type + local name).
- **Input variables** are declared with type/default and read as `var.name`. Values can come from default, TF_VAR_ environment variable, terraform.tfvars, or `-var` on the command line (highest priority).

- **Output variables** print values after apply (here the public IP) and can feed other modules.

4. Commands and their effect [4 M]

Command	Effect on this scenario
terraform init	Downloads the AWS provider plugin, initialises backend/modules, creates local state setup.
terraform validate	Checks syntax; silent/Success if valid, otherwise shows file and line of the error.
terraform plan	Preview: **1 to add** (the instance). Nothing is created yet.
terraform apply -var="instance_type=t2.small"	Creates the EC2 instance after you type **yes**; overrides the default type; prints web_public_ip; updates state.
terraform destroy	Deletes the instance (confirm with yes) - avoids unwanted cloud cost.

5. Conclusion [2 M]

Terraform turns the server request into repeatable, reviewable code. Variables make it reusable across environments, outputs expose results, and the plan-apply cycle with a state file guarantees only the needed changes are made.

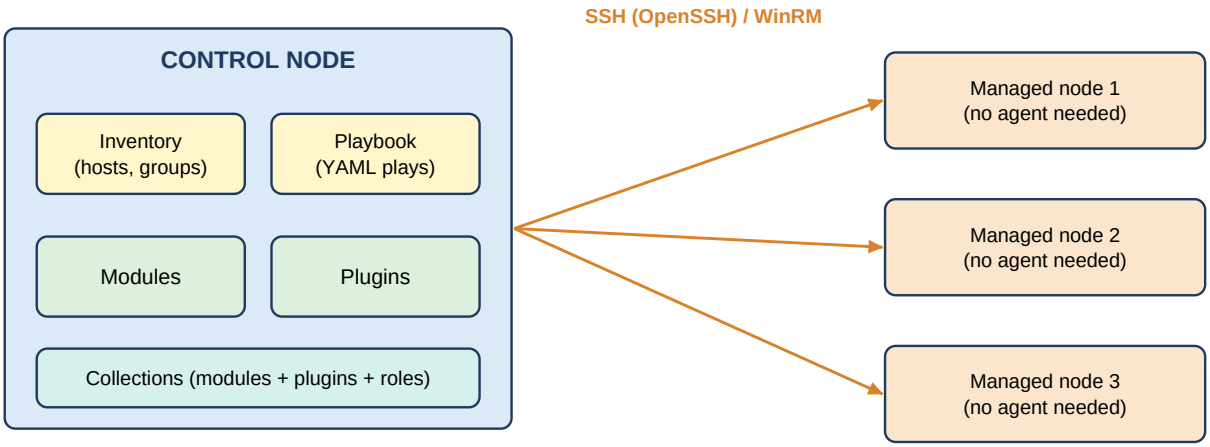
QUESTION 3 | 15 MARKS | UNIT 3 - Ansible

A company must install and configure Nginx on 50 Ubuntu web servers and restart it only when the configuration changes. Design the Ansible solution: inventory, playbook, execution and key Ansible features used.

1. Introduction and need [2 M]

Ansible (Red Hat, Python, YAML) is an open-source, **agentless** automation tool for configuration management and application deployment. Doing this by hand on 50 servers is slow and inconsistent; Ansible needs only SSH, uses readable playbooks and is **idempotent**.

2. Architecture diagram [3 M]



Modules are copied over SSH, executed, then removed

Fig 3.5 Ansible agentless architecture

3. Step 1 - Inventory [2 M]

```
# inventory.ini
[webservers]
web1 ansible_host=192.168.1.11
web2 ansible_host=192.168.1.12
# ... web50

[webservers:vars]
ansible_user=ubuntu
```

Verify: `ansible-inventory -i inventory.ini --list` and `ansible webservers -i inventory.ini -m ping`. For many hosts a YAML or dynamic inventory is preferable.

4. Step 2 - Playbook [4 M]

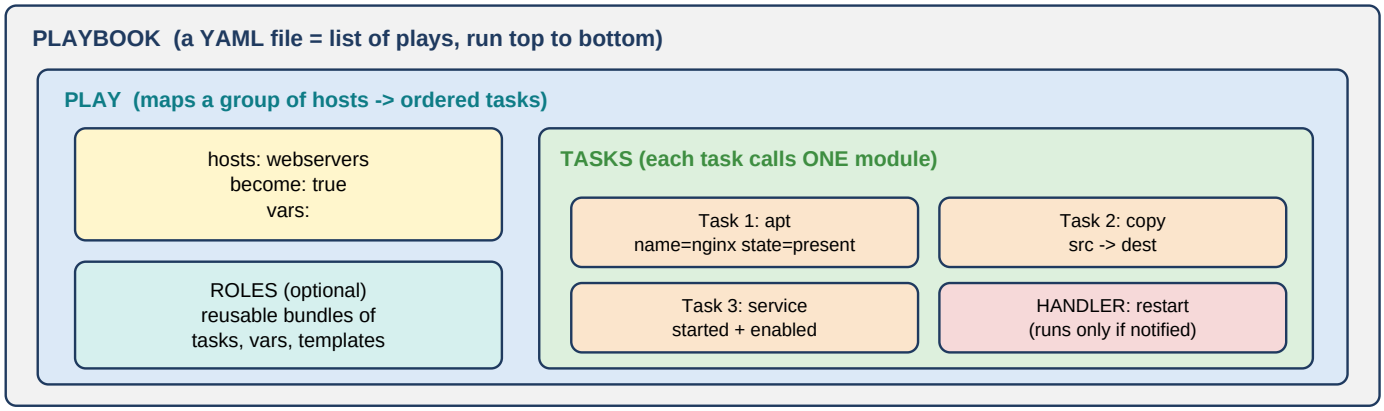


Fig 3.6 Playbook > Play > Task > Module hierarchy

```
---
- name: Configure web servers
  hosts: webservers
  become: true
  tasks:
    - name: Install nginx
      ansible.builtin.apt:
        name: nginx
        state: present
        update_cache: true
    - name: Copy nginx config
      ansible.builtin.template:
        src: nginx.conf.j2
        dest: /etc/nginx/nginx.conf
      notify: Restart nginx          # only if this task changed something
    - name: Ensure nginx is running
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: true
  handlers:
    - name: Restart nginx
      ansible.builtin.service:
        name: nginx
        state: restarted
```

5. Step 3 - Execution and features [3 M]

- Dry run: `ansible-playbook -i inventory.ini site.yml --check`; real run without --check. Tasks execute on many hosts in parallel (forks).
- **Idempotency:** on a second run the report shows *ok* and *changed=0*; the handler does not run because nothing changed.
- **Handler** restarts Nginx only when the configuration task reports a change - exactly the requirement.
- **Modules used:** apt, template, service - each task calls one module executed on the managed node.

6. Conclusion [1 M]

A single inventory and playbook configure all 50 servers consistently, with no agents, safe re-runs and restarts only when needed.

QUESTION 4 | 15 MARKS | UNIT 3 - Modules, Plugins, Collections

An organisation manages AWS instances and Cisco switches using Ansible. Explain how modules, plugins and collections help, and show how a collection is installed and used in a playbook.

1. Definitions [4 M]

Concept	Definition	Runs on
Module	Code Ansible copies to a managed node and executes to do one task (apt, copy, ec2_instance).	Managed node
Plugin	Code that extends Ansible core - connection (how to connect), filter (manipulate data), callback (console output), lookup, inventory.	Control node
Collection	Distribution format bundling playbooks, roles, modules and plugins; named namespace.collection; installed via Ansible Galaxy.	Installed on control node

2. Diagram [3 M]

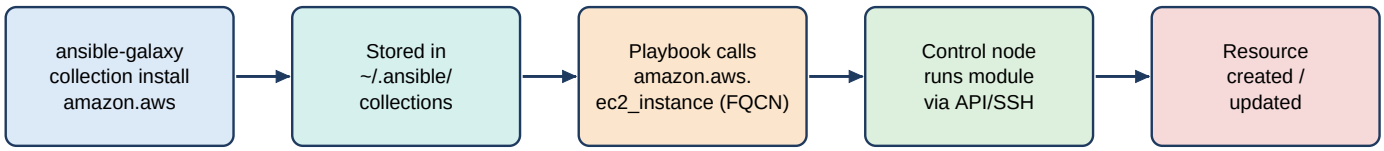


Fig 3.7 Using an Ansible collection in a playbook

3. Application to the scenario [4 M]

- **AWS:** collection **amazon.aws** gives modules such as ec2_instance and the **aws_ec2 inventory plugin**, which builds the host list dynamically from AWS.
- **Cisco switches:** collection **cisco.ios** provides ios_config; the connection plugin **network_cli** (from ansible.netcommon) connects to network devices instead of normal SSH.
- **Callback and filter plugins** give readable output and data transformation in templates.
- Collections are versioned and installed separately, so new vendor support arrives without upgrading Ansible core.

4. Installation and use in a playbook [4 M]

```
# install
ansible-galaxy collection install amazon.aws cisco.ios
ansible-galaxy collection install -r requirements.yml     # or from a file

# playbook using FQCN (Fully Qualified Collection Name)
---
- name: Manage cloud and network
  hosts: localhost
  tasks:
    - name: Launch EC2 instance
      amazon.aws.ec2_instance:        # namespace.collection.module
        name: demo-web
        instance_type: t2.micro
        image_id: ami-0c55b159cbfafe1f0
        region: us-west-2
        state: running

- name: Configure switches
  hosts: switches
  gather_facts: false
  tasks:
    - name: Set banner
      cisco.ios.ios_config:
        lines:
          - banner motd "Authorised users only"
```

Alternative: list collections under the `collections:` keyword and use short module names. FQCN is recommended because it avoids name clashes.

5. Conclusion [1-line closing]

Modules do the work, plugins extend how Ansible works, and collections package both so one tool manages cloud and network devices cleanly.

QUESTION 5 | 15 MARKS | UNIT 4 - Docker

A developer's application runs on the development laptop but crashes on the test server because of missing libraries. Explain how Docker solves this problem. Describe Docker architecture and the workflow from code to running container.

1. Problem and Docker introduction [2 M]

The failure is an **environment mismatch** ("works on my machine"): different library versions on dev and test servers. **Docker** is an open-source containerisation platform that packs the application with all its dependencies into a **container**, so it runs identically anywhere Docker is installed.

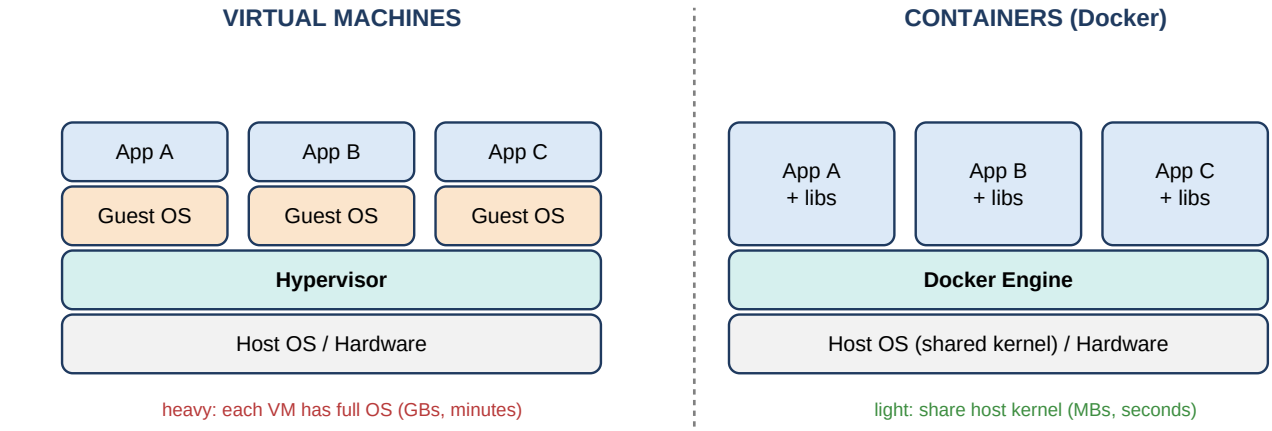


Fig 4.1 Virtual machines vs containers

2. Docker architecture [4 M]

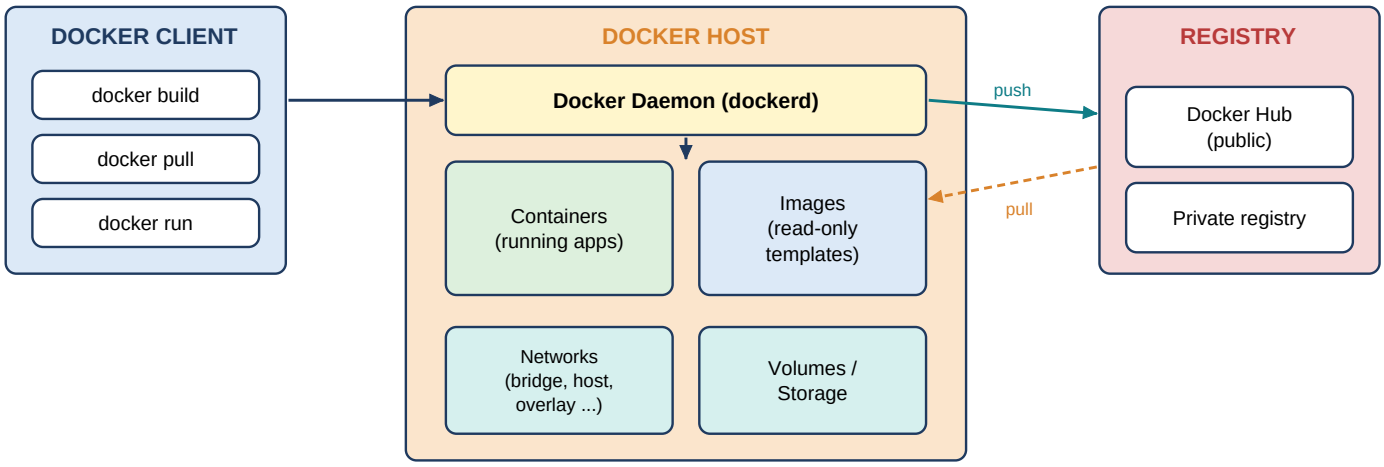


Fig 4.2 Docker architecture - client, host (daemon + objects) and registry

- **Client** - user interface; docker commands call the Docker API.
- **Daemon** - manages images, containers, networks, volumes; client and daemon talk via REST API over UNIX socket/network.
- **Host** - machine running the daemon and containers.
- **Registry** - stores images (Docker Hub public; private registries possible); pull downloads, push uploads.

3. Workflow from code to container [5 M]

Fig 4.3 From Dockerfile to a running container (docker push/pull move images to/from a registry)

```
# Dockerfile (inside project)
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

Step	Command	Result
Build image (dev)	docker build -t myapp:1.0 .	Read-only image with exact libraries
Publish	docker push user/myapp:1.0	Image stored in registry
Fetch (test server)	docker pull user/myapp:1.0	Same image downloaded
Run	docker run -d -p 8080:80 --name web user/myapp:1.0	Container runs identically; port mapped
Persist data	docker run -v mydata:/data ...	Volume keeps data after container removal

Networking: containers join the default **bridge** network (other drivers: host, overlay, none, macvlan). Storage: data volumes, volume containers, directory mounts, storage plugins.

4. Benefits and conclusion [4 M]

- **Portability** - one image for dev, test and prod.
- **Light weight** - shares host kernel; starts in seconds.
- **Isolation** - containers do not interfere.
- **Efficient resource use** and fast CI/CD delivery.

Conclusion: packaging the app and its dependencies as an image removes environment differences, so the test-server failure cannot recur.

QUESTION 6 | 15 MARKS | UNIT 4 - Docker Swarm

An e-commerce site must stay available if one server fails and must scale to 5 replicas of its web service. Design the solution using Docker Swarm and explain how failures and updates are handled.

1. Introduction [2 M]

Docker Swarm is Docker's native clustering and orchestration mode. It joins several Docker hosts into one cluster so containers can be deployed, scaled, load-balanced and healed automatically according to a **desired state**.

2. Diagram [4 M]

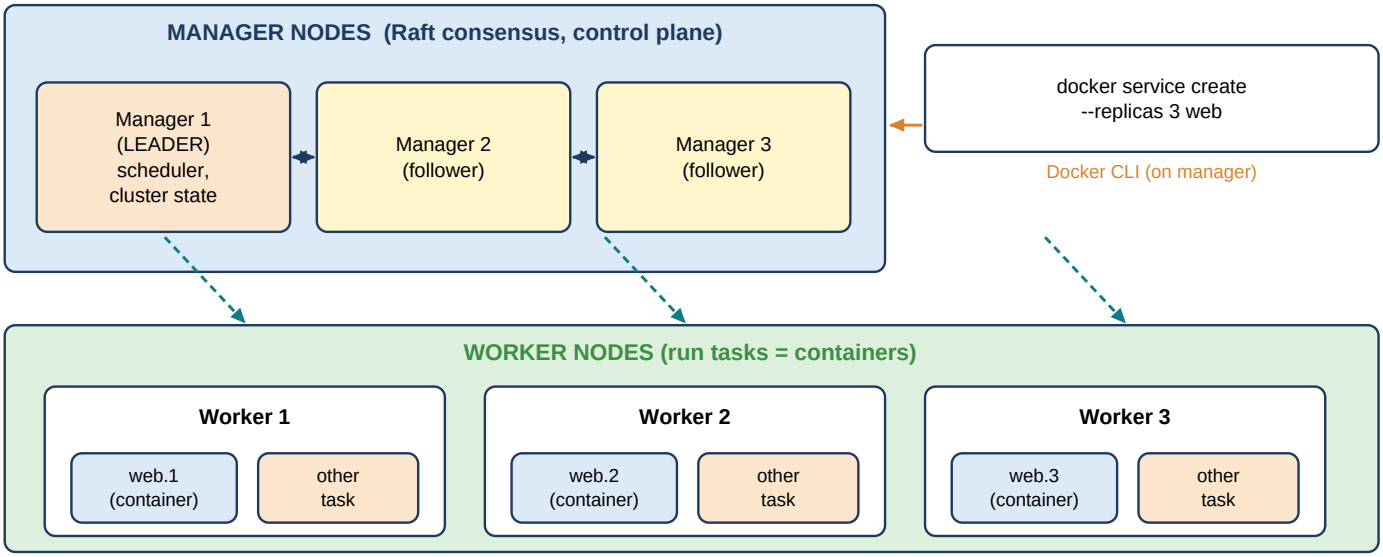


Fig 4.5 Docker Swarm cluster - dashed arrows = task assignment; overlay network + routing mesh connect all nodes

3. Components [3 M]

Component	Role
Manager nodes	Keep cluster state using Raft consensus, schedule tasks; 3 or 5 managers (odd) for fault tolerance; one leader.
Worker nodes	Run the tasks (containers) assigned by managers.
Service / task	Service = desired state (image, replicas, ports); task = one container instance.
Overlay network + routing mesh	Containers talk across hosts; published port reachable on any node.

4. Implementation commands [3 M]

```
docker swarm init --advertise-addr 192.168.1.10          # manager 1
docker swarm join --token <TOKEN> 192.168.1.10:2377       # on each worker / manager
docker node ls                                            # verify nodes

docker service create --name web --replicas 3 -p 80:80 nginx
docker service scale web=5                                  # scale to 5 replicas
docker service update --image nginx:1.25 web                # rolling update
docker service rollback web                                 # undo if problems
```

5. Failure and update handling [2 M]

- **Worker/container failure:** the manager detects missing tasks and starts replacements on healthy nodes, restoring 5 replicas (self-healing).
- **Manager failure:** remaining managers elect a new leader by Raft majority - hence an odd number of managers.
- **Rolling update:** tasks are replaced a few at a time, so the site stays online.

6. Conclusion [1 M]

Swarm gives high availability, simple scaling and built-in load balancing with Docker-native tools; for very large or complex workloads Kubernetes is preferred.

QUESTION 7 | 15 MARKS | UNIT 4 - Kubernetes Architecture

Explain Kubernetes cluster architecture with a neat diagram and describe what happens internally when "kubectl apply -f pod.yaml" is executed.

1. Introduction [2 M]

Kubernetes is an open-source container orchestration platform (originally from Google) that automates deployment, scaling and operation of containerised applications. It is **declarative**: you submit desired state and controllers continuously make the cluster match it, giving self-healing, scaling and rolling updates.

2. Cluster architecture diagram [4 M]

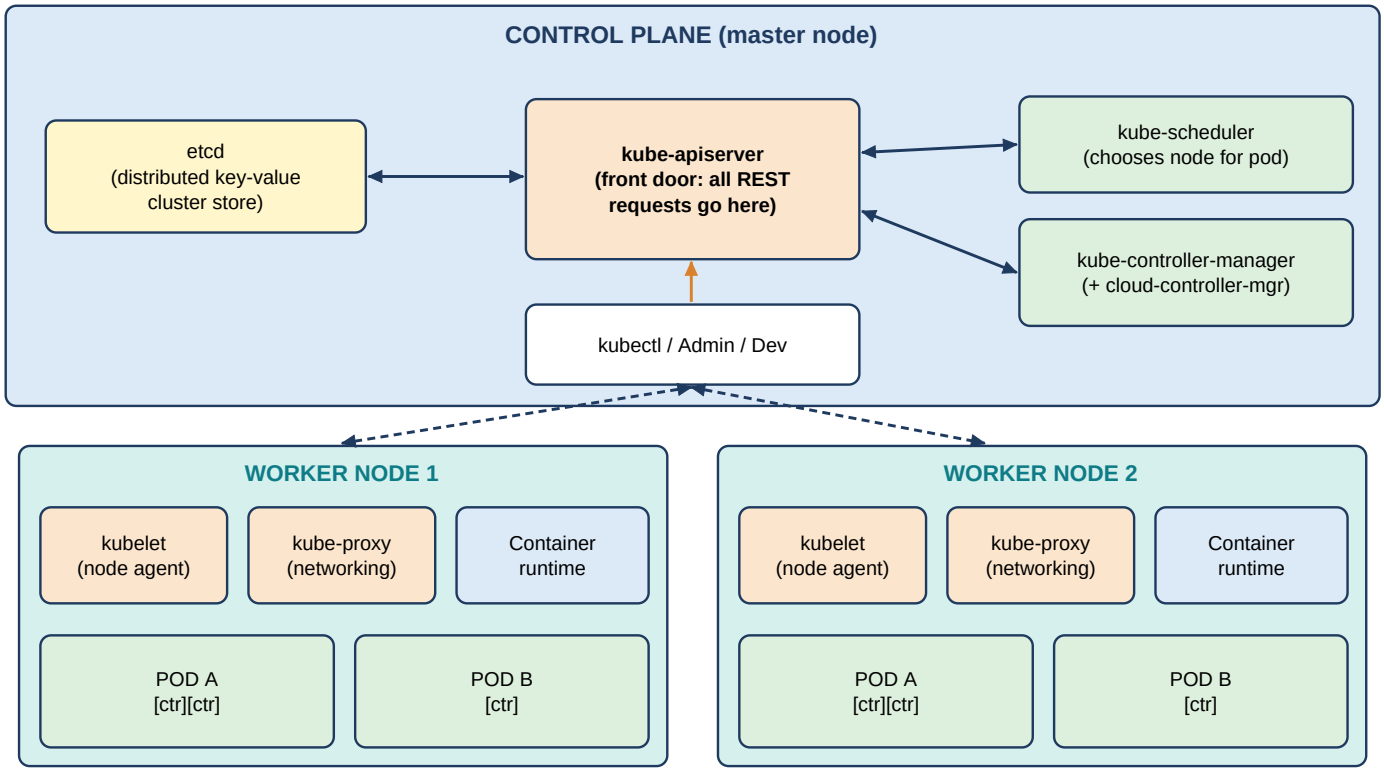


Fig 4.6 Kubernetes cluster architecture (control plane + worker nodes)

3. Components [4 M]

Where	Component	Function
Control plane	kube-apiserver	Front end; receives all REST requests; only component talking to etcd.
	etcd	Distributed key-value store holding all cluster data.
	kube-scheduler	Chooses the best node for new pods by resources and policy.
	controller-manager	Runs controllers (replication, endpoints, node) to keep actual state = desired state; cloud-controller-manager handles cloud specifics.
Worker node	kubelet	Node agent; runs pods as specified and reports health.
	kube-proxy	Maintains network rules; routes service traffic to pods.
	Container runtime	Runs containers (containerd, CRI-O).

4. What happens on kubectl apply [4 M]

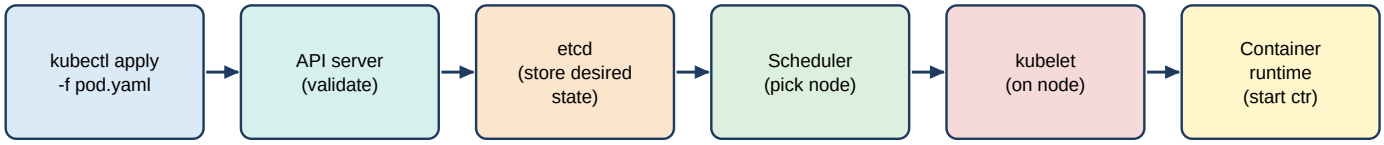


Fig 4.7 What happens when you create a pod

- **kubectl** converts the command into a REST call to the API server.
- API server authenticates, validates and stores the Pod object (desired state) in **etcd**.
- **Scheduler** sees an unscheduled pod and binds it to the best-fit node.
- **kubelet** on that node notices the assignment and tells the container runtime to pull the image and start containers.
- kubelet reports status back; controllers and kube-proxy keep the pod and its service rules correct.

5. Conclusion [1 M]

The control plane decides and records, the worker nodes execute; this separation lets Kubernetes heal, scale and update applications automatically.

QUESTION 8 | 15 MARKS | UNIT 4 - Design of Pods

A web application needs an Nginx container and a log-shipping container that share log files. Design the pod, run it with three replicas, expose it to users and perform a zero-downtime update.

1. Introduction to pods [2 M]

A **Pod** is the smallest deployable unit: one or more tightly coupled containers sharing one **IP address**, network namespace and volumes. Pods are **ephemeral**, so they are managed by a Deployment and reached through a Service.

2. Pod design diagram [3 M]

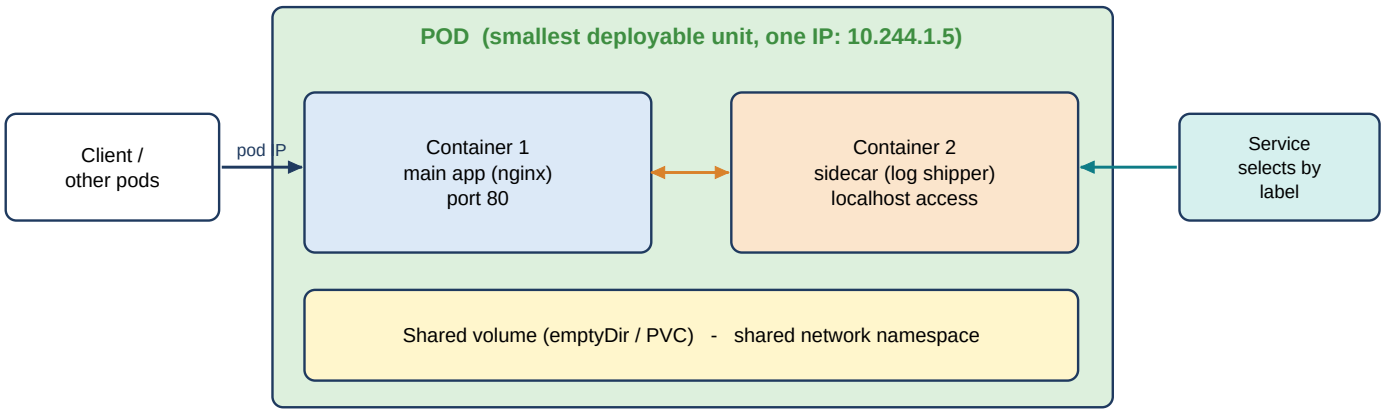


Fig 4.8 Multi-container pod - shared network and storage

- Two containers: **web** (main) and **log-shipper** (sidecar pattern); they reach each other on `localhost` and share an **emptyDir** volume for logs.
- Good design: group containers only if they must share lifecycle, network and storage; add readiness/liveness probes and resource limits.

3. Pod and Deployment definition [4 M]

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 3
  selector:
    matchLabels: { app: web }
  template:
    metadata:
      labels: { app: web }
    spec:
      volumes:
      - name: logs
        emptyDir: {}
      containers:
      - name: web
        image: nginx:1.24
        ports: [{ containerPort: 80 }]
        readinessProbe: { httpGet: { path: /, port: 80 } }
        volumeMounts: [{ name: logs, mountPath: /var/log/nginx }]
      - name: log-shipper
        image: busybox
        command: ["sh","-c","tail -F /logs/access.log"]
        volumeMounts: [{ name: logs, mountPath: /logs }]
```

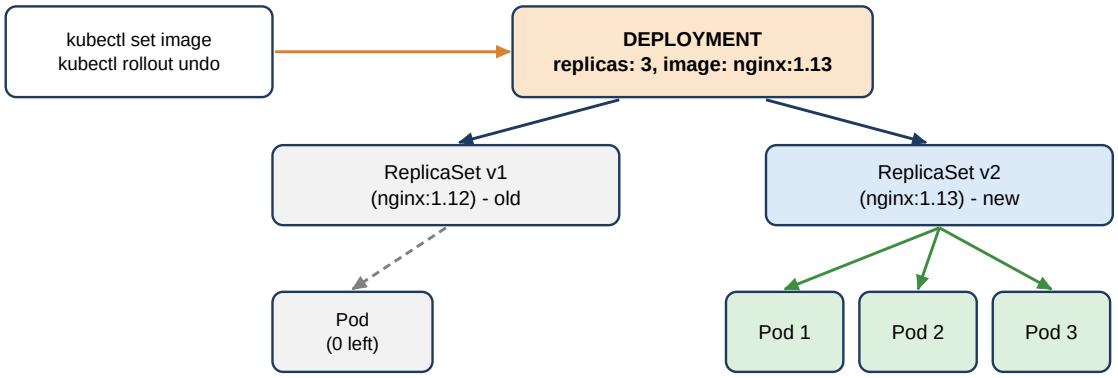


Fig 4.9 Deployment manages ReplicaSets, which manage Pods (rolling update and rollback)

4. Exposing the application [3 M]

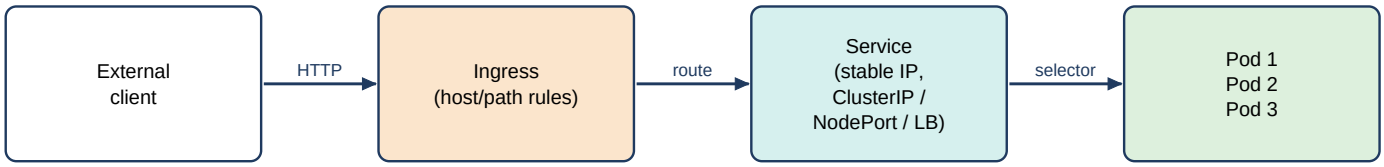


Fig 4.10 Exposing pods: client -> Ingress -> Service -> Pods

```
apiVersion: v1
kind: Service
metadata:
  name: web
spec:
  type: LoadBalancer            # or NodePort; use Ingress for host/path routing
  selector: { app: web }
  ports: [{ port: 80, targetPort: 80 }]
```

5. Zero-downtime update and scaling [3 M]

Task	Command
Deploy	kubectl apply -f web.yaml
Rolling update	kubectl set image deployment web web=nginx:1.25
Watch progress	kubectl rollout status deployment web
Rollback if needed	kubectl rollout undo deployment web
Autoscale	kubectl autoscale deployment web --min=3 --max=6 --cpu-percent=50

The Deployment creates a new ReplicaSet and gradually moves pods to it (RollingUpdate); the **readiness probe** stops traffic going to pods that are not ready, so users see no downtime.

6. Conclusion [1-line closing]

A sidecar pod with shared volume, managed by a Deployment and exposed through a Service, gives a scalable, self-healing and easily updated web tier.